

Electricity Demand Forecasting Using AI

Project Lab Automation

Anton Valentin Dilg, Ramin Leon Neymeyer, Ramon Rennert

January 25, 2026

Technische Universität Berlin

Problems:

- Better forecasts reduce imbalance penalties and operational inefficiencies [1]
- Increasing variability from renewables makes accurate planning more challenging

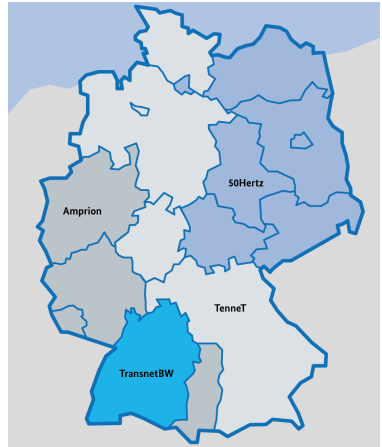


Figure 1: Transmission system operators in Germany and their control areas [2].

Objectives:

- Build a complete dataset
- Forecast the hourly demand profile for the following day

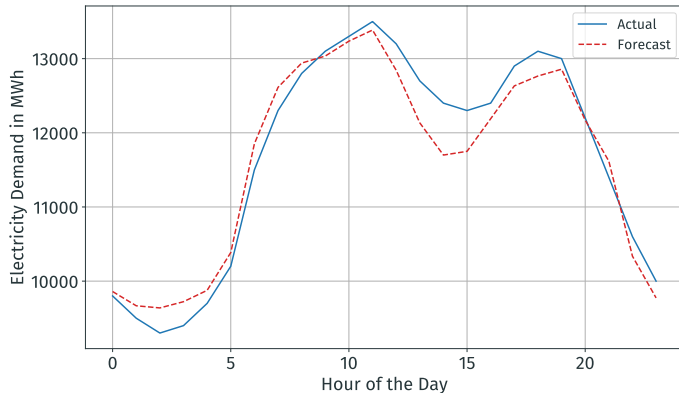


Figure 2: Example of hourly electricity demand and forecast.

Time range: 01/01/2015–12/31/2024 with hourly resolution

Power Sources: From the Federal Network Agency, freely available (CC BY 4.0) [2]

Weather: Five different stations across region using Meteostat [3]

Temporal Features: School holidays across all federal states [4]

Market Data: Day-ahead Price in DE/(AT)/LU bidding zone [5]

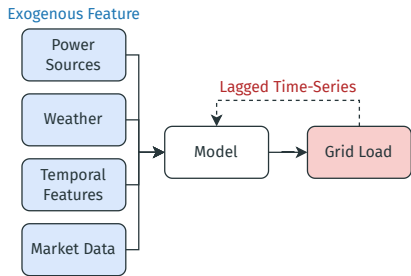
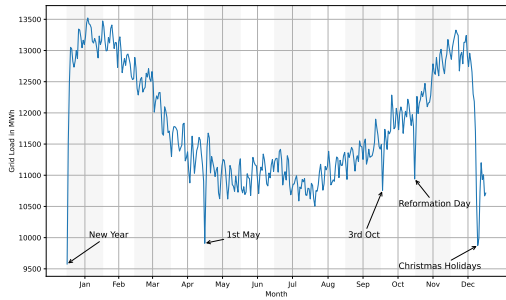
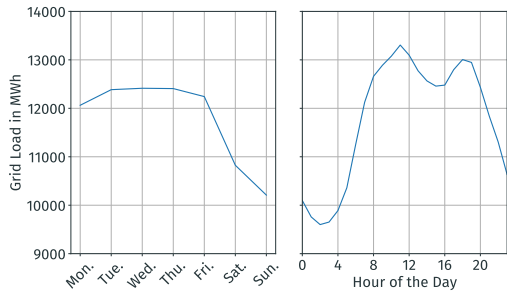


Figure 3: Forecasting model with exogenous features and lagged targets.



(a) Average Annual Course of Grid Load.



(b) Average Grid Load by Day and by Hour.

Figure 4: Temporal patterns of average grid load.

Backtesting

Backtesting setup:

- Forecast horizon: 24 hours
- Model is not refitted at each fold
- Validation period: average MAPE in 01/01/2024–12/31/2024

Metric	Unit	Description
MAE	MWh	Robust to outliers symmetric error measure
MSE	MWh ²	Penalizes large errors more strongly for detecting instability in forecasts
MAPE	%	Relative error measure comparable across different demand levels [6]

Baseline Model

Seasonal Naive Forecaster uses equivalent point in the past “season”, e.g. the same hour on the previous day [7]

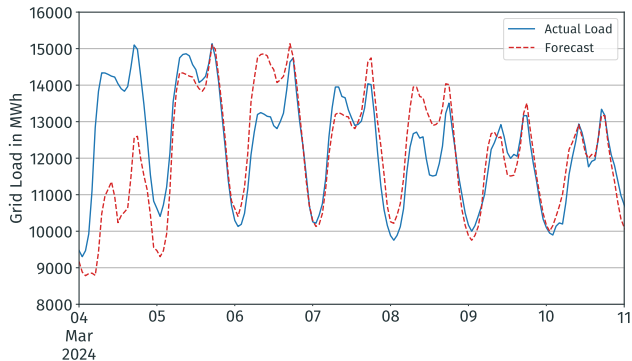
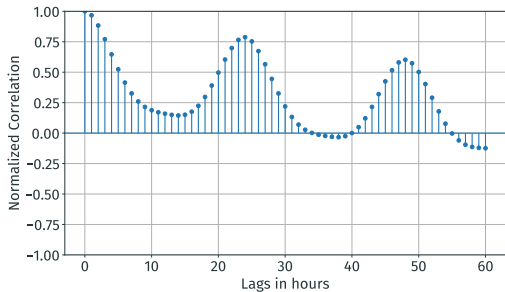
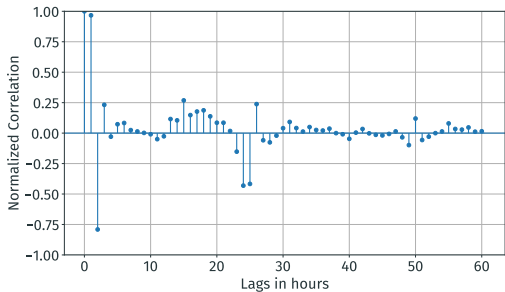


Figure 5: Baseline model with MAPE of 8.57%.

Forecasting Model – Autocorrelation



(a) Autocorrelation.



(b) Partial Autocorrelation.

Figure 6: Autocorrelation and partial autocorrelation of the grid load time series.

Forecasting Model – Recursive Forecaster

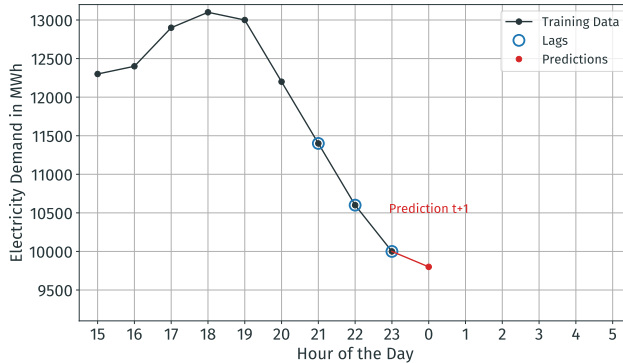


Figure 7: Recursive multi-step forecasting.

Forecasting Model – Recursive Forecaster

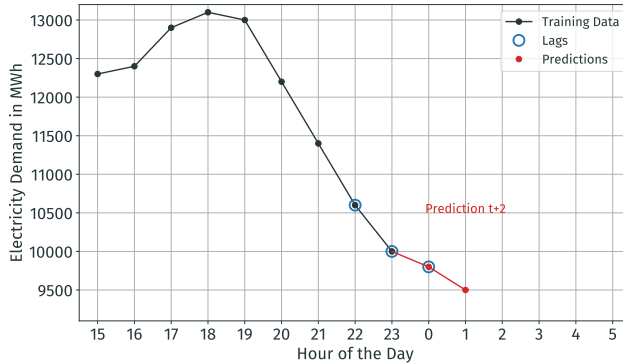


Figure 7: Recursive multi-step forecasting.

Forecasting Model – Recursive Forecaster

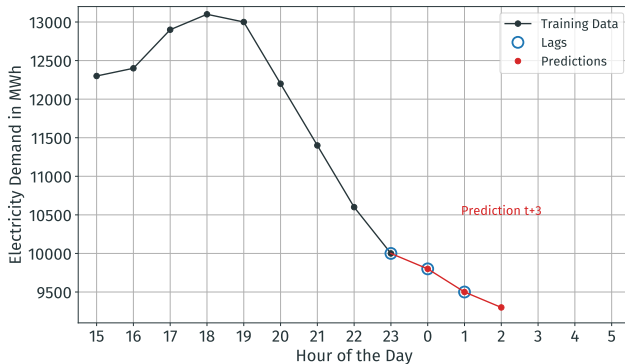


Figure 7: Recursive multi-step forecasting.

Forecasting Model – Recursive Forecaster

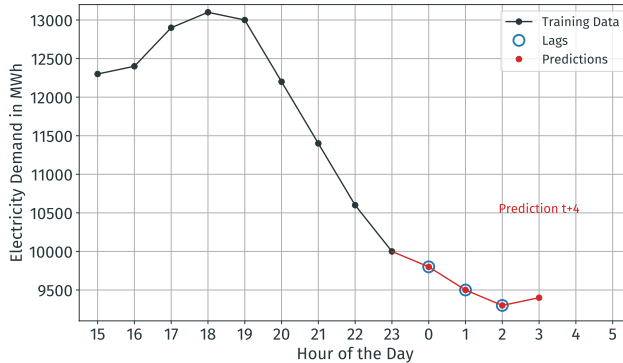


Figure 7: Recursive multi-step forecasting.

Forecasting Model – Recursive Forecaster

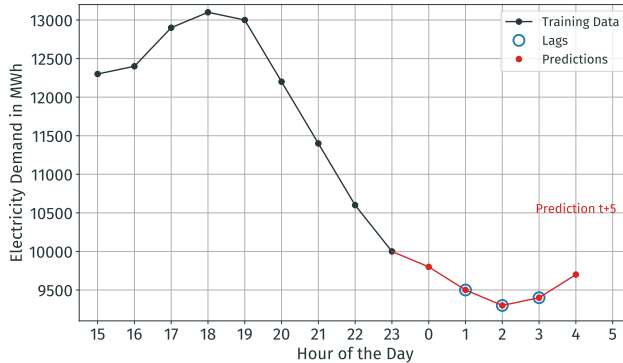


Figure 7: Recursive multi-step forecasting.

LightGBM

- Histogram-based splitting for high efficiency
- Leaf-wise tree growth strategy

XGBoost

- Strong regularization to reduce overfitting
- Level-wise tree growth for stable training

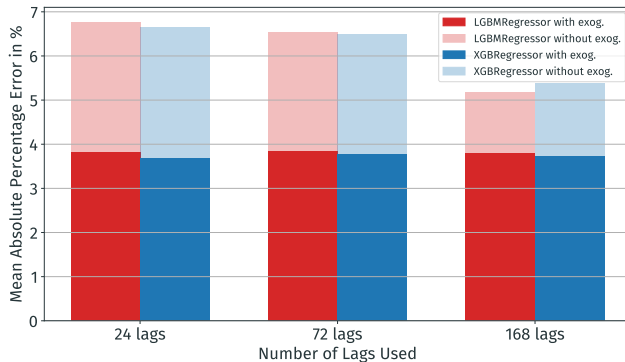


Figure 8: Estimator comparison with exogenous features and different lags.

Bayesian Optimization with 50 trials to efficiently explore the hyperparameter space.

Computational Cost:

- XGBoost: ~83 min
- LightGBM: ~78 min

Performance Gain:

- XGBoost: MAPE from 3.68% to 3.09%
- LightGBM: MAPE from 3.81% to 3.11%

Final Model

Model is refitted at each iteration to reflect real-world production behavior

Backtesting time with refitting increases up to 3.5h (with 2 CPU Cores)

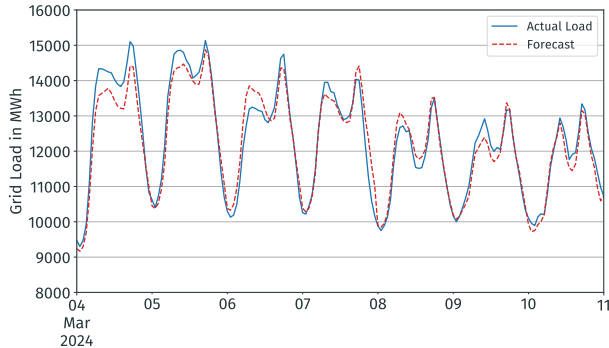


Figure 9: Tuned LGBMRegressor with a MAPE of 2.91%.

Feature Importance

Two complementary measures are considered:

- **Split importance:** number of times a feature is used for splitting
- **Gain importance:** total reduction of the loss function achieved by splits using the feature

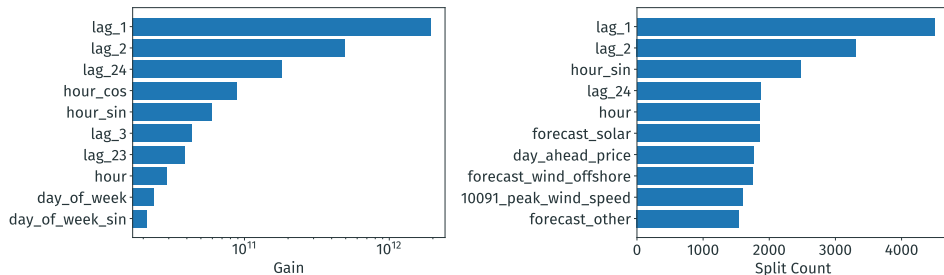
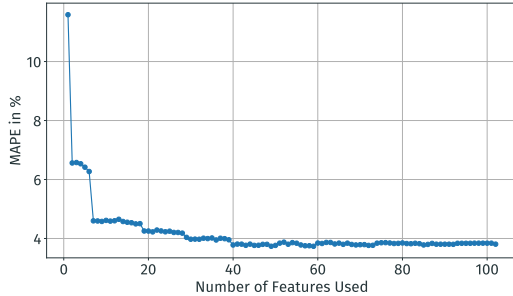
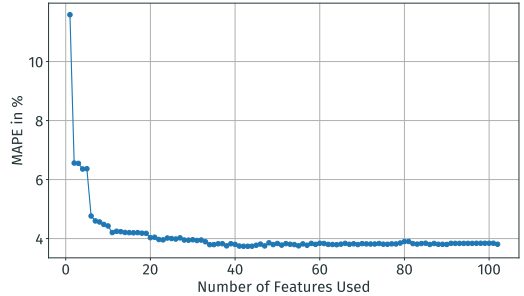


Figure 10: Feature importance of forecasting model.

Feature Importance



(a) Total Gain.



(b) Split count.

Figure 11: MAPE as a function of the number of selected features using rank-based feature addition.

- Optimize the number of features by balancing runtime and prediction accuracy
- Explore probabilistic forecasting approaches to better quantify uncertainty
- Investigate deep learning-based models for capturing more complex temporal patterns

Questions or Remarks?

References i

- [1] A. Miraki, P. Parviainen, and R. Arghandeh, *Electricity demand forecasting at distribution and household levels using explainable causal graph neural network*, *Energy and AI*, vol. 16, p. 100 368, Apr. 2024. DOI: <https://doi.org/10.1016/j.egyai.2024.100368>.
- [2] Bundesnetzagentur, *SMARD – Strommarktdaten für Deutschland*, smard.de, Accessed: Nov. 13, 2025, 2025.
- [3] C. S. Lamprecht, *Meteostat Python*, meteostat.net, Accessed: November 12, 2025, 2020.
- [4] M. N. Johannsen and C. Sanders, *Schulferien und Feiertage in Deutschland*, schulferien.org, Accessed: Decemeber 01, 2025, 2001.
- [5] Burger, Bruno, *Energy-Charts API Documentation*, Accessed: 2026-01-16.
- [6] M. Fotakis, *The Validity of Backtesting for Evaluation of Autoregressive Time Series Predictions*, M.S. thesis, Tilburg University, 2021.

- [7] J. Amat Rodrigo and J. Escobar Ortiz, *skforecast*, version 0.19.1, Dec. 2025. DOI: [10.5281/zenodo.8382788](https://doi.org/10.5281/zenodo.8382788).
- [8] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*. New York, NY, USA: Springer New York Inc., 2001. DOI: [10.1007/b94608](https://doi.org/10.1007/b94608).
- [9] Y. Shi, G. Ke, D. Soukhavong, J. Lamb, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, et al., *lightgbm: Light Gradient Boosting Machine*, 2025.

Appendix – Gradient Boosting Machine

Fitting Algorithm [8]:

1. Start with simple model, e.g. mean of target variable

$$F_1(x) = f_1(x) = \bar{y}$$

2. Calculate error gradient

$$\hat{r}_{mi} = - \left. \frac{\partial \mathcal{L}(y_i, F(x_i))}{\partial F(x_i)} \right|_{F=F_m} \quad \forall i \in \{1, \dots, N\}$$

3. Fit a new weak learner to the residuals

$$f_{m+1}(x) \approx \hat{r}_m$$

4. Determine how much of the new learner to add

$$\gamma_{m+1} = \arg \min_{\gamma} \sum_{i=1}^N \mathcal{L}(y_i, F_m(x_i) + \gamma f_{m+1}(x_i))$$

5. Update the model

$$F_{m+1}(x) = F_m(x) + \gamma_{m+1} f_{m+1}(x)$$

LightGBM uses leaf-wise tree growth, leaf with the highest loss reduction is split [9]

Appendix – SHapley Additive exPlanations Values

- SHAP values quantify how much each feature contributes to an individual prediction compared to a baseline.
- They are based on concepts from cooperative game theory and provide consistent and interpretable feature attributions.

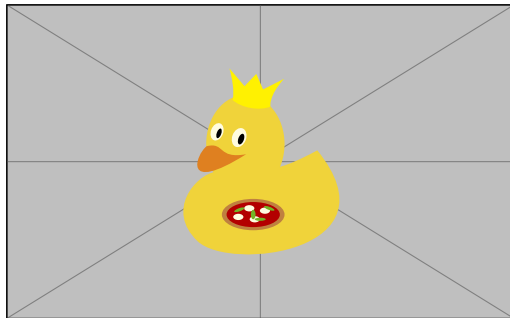


Figure 12: SHAP values for the LGBM estimator